AWS NETWORK ASSOCIATE

AWS VPC Peering

Project Report

Connect two VPCs and prove private connectivity with EC2, custom route tables, security groups and a bastion host.



Name : YASIR SHARIEF QURASHI

Batch : EWB AWS INTERN

Region : Mumbai / ap-south-1

Completion date: 25/10/2026

00 / SUBMISSION CHECKLIST

Report contents

Every item from the submission checklist, with where to find it

Checklist item	Page
Cover: name, batch, project title, Region, completion date	1
Architecture diagram and inventory with actual resource IDs and private IPs	3
Subnet CIDRs and all three route tables with subnet associations	6
Active peering connection and inbound rules for each security group	8
Baseline failure, forward and reverse ping success, App-B SSH proof	11
Controlled ICMP failure and recovery, with a short explanation	14
Cleanup evidence plus a short note on what I learned	16

Project summary

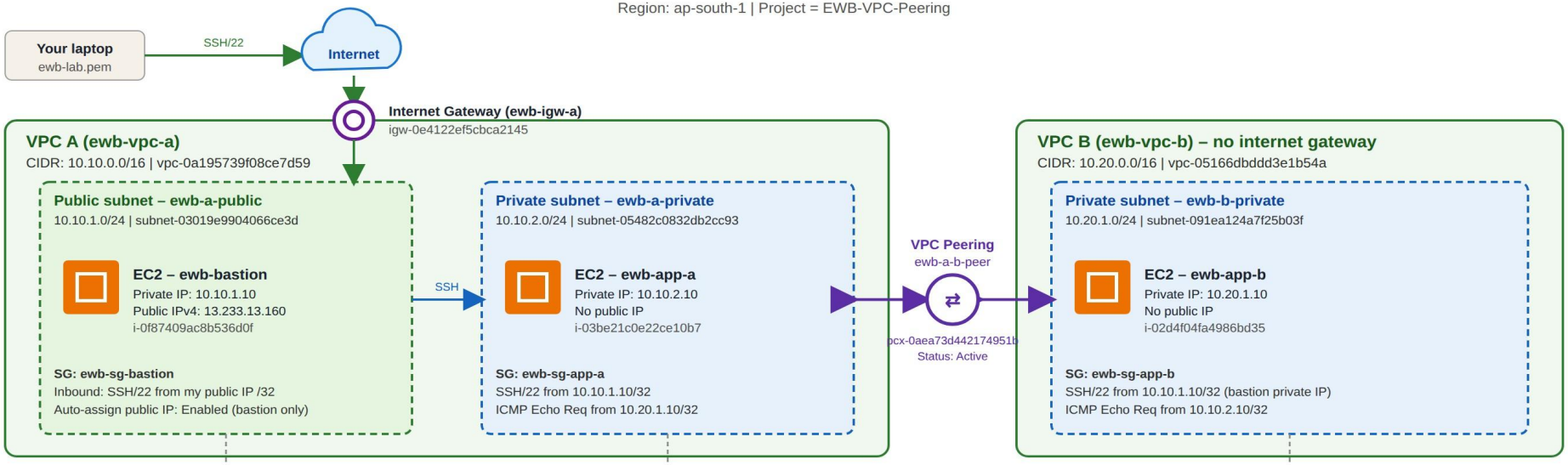
I built two VPCs in **ap-south-1** (Mumbai, Availability Zone ap-south-1a) with non-overlapping CIDRs (10.10.0.0/16 and 10.20.0.0/16), joined them with a VPC peering connection and tested private connectivity between App-A (10.10.2.10) and App-B (10.20.1.10) using their private IPv4 addresses. A single public bastion in VPC A is the only internet-facing host; both application servers have no public IP. Administration uses SSH ProxyJump through the bastion, so the private key stays on my laptop. All three instances are Amazon Linux 2023 on t3.micro.

Implemented **AWS VPC Peering for secure private communication between two Amazon Virtual Private Clouds (VPCs)** in the Mumbai (`ap-south-1`) region. The project uses a public bastion host for secure SSH administration of private EC2 instances, custom route table
Technologies: Amazon VPC, EC2, VPC Peering, Linux, SSH ProxyJump, Route Tables, Security Groups, ICMP, AWS Networking.

01 / ARCHITECTURE

AWS VPC Peering – Implemented Architecture

Region: ap-south-1 | Project = EWB-VPC-Peering



Route Table – ewb-rt-a-public (rtb-041f1ac148cb6e157)

Destination	Target
10.10.0.0/16	local
0.0.0.0/0	igw-0e4122ef5cbca2145
10.20.0.0/16	pcx-0aea73d442174951b

Route Table – ewb-rt-a-private (rtb-00763f404aa0caec3)

Destination	Target
10.10.0.0/16	local
10.20.0.0/16	pcx-0aea73d442174951b

Route Table – ewb-rt-b-private (rtb-0a071720960f64016)

Destination	Target
10.20.0.0/16	local
10.10.0.0/16	pcx-0aea73d442174951b

Traffic flow

- Laptop → Internet → IGW → Bastion (public subnet, SSH/22)
- Bastion → App-A (within VPC A, local route) and Bastion → App-B (ProxyJump via peering)
- App-A (10.10.2.10) ↔ App-B (10.20.1.10) over VPC peering using private IPs

Return traffic is bidirectional: each side has a route to the other VPC CIDR via the pcx.
Peering does not give VPC B internet access through VPC A (non-transitive).

Architecture diagram and inventory

Actual resource IDs and private IPs from my AWS account.

Resource inventory

Resource	Name tag	ID	CIDR / address
VPC A	ewb-vpc-a	vpc-0a195739f08ce7d59	10.10.0.0/16
VPC B	ewb-vpc-b	vpc-05166dbddd3e1b54a	10.20.0.0/16
Public subnet	ewb-a-public	subnet-03019e9904066ce3d	10.10.1.0/24
Private subnet	ewb-a-private	subnet-05482c0832db2cc93	10.10.2.0/24
Private subnet	ewb-b-private	subnet-091ea124a7f25b03f	10.20.1.0/24
Internet gateway	ewb-igw-a	igw-0e4122ef5cbca2145	Attached to VPC A
Peering	ewb-a-b-peer	pcx-0aea73d442174951b	VPC A to VPC B
Route table	ewb-rt-a-public	rtb-041f1ac148cb6e157	VPC A
Route table	ewb-rt-a-private	rtb-00763f404aa0caec3	VPC A
Route table	ewb-rt-b-private	rtb-0a071720960f64016	VPC B
Security group	ewb-sg-bastion	sg-0b053bcdfa5169dc9	VPC A
Security group	ewb-sg-app-a	sg-0c9ff9c451e186b1a	VPC A
Security group	ewb-sg-app-b	sg-07d3c238f130a7e4d	VPC B
EC2 bastion	ewb-bastion	i-0f87409ac8b536d0f	10.10.1.10 + public 13.233.13.160
EC2 App-A	ewb-app-a	i-03be21c0e22ce10b7	10.10.2.10 (no public IP)
EC2 App-B	ewb-app-b	i-02d4f04fa4986bd35	10.20.1.10 (no public IP)

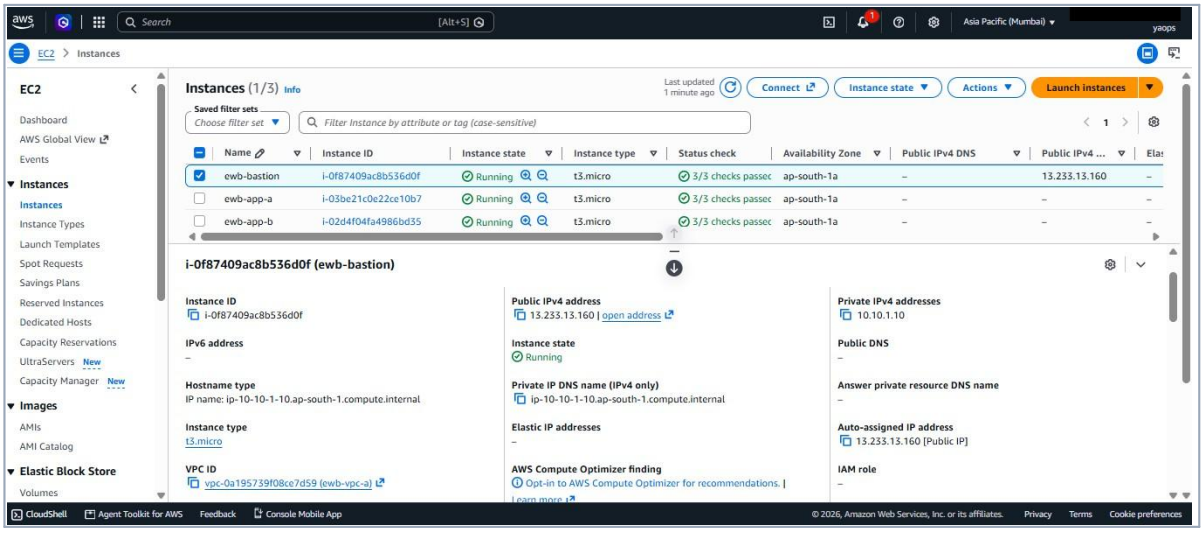
01b / BUILD EVIDENCE

Instance and internet gateway evidence

Console screenshots from my account.

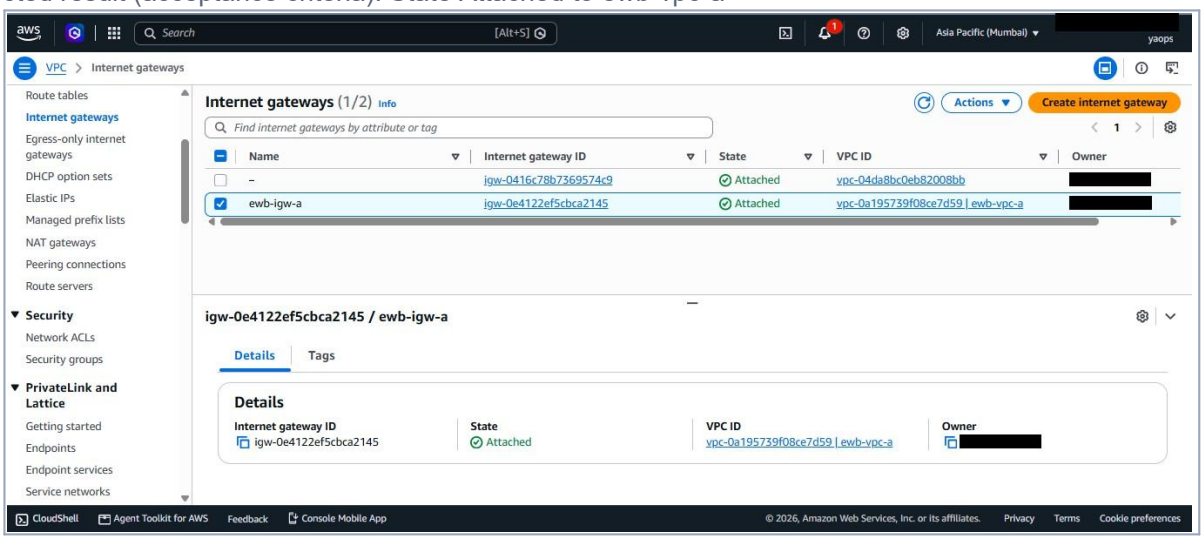
Host: EC2 console | Test: ewb-bastion instance details

Expected result (acceptance criteria): Running, 3/3 checks, private IP 10.10.1.10, public IPv4 assigned



Host: VPC console | Test: Internet gateway ewb-igw-a

Expected result (acceptance criteria): State Attached to ewb-vpc-a



02 / ROUTING

Subnet CIDRs and route tables

Each route table is explicitly associated with its subnet. Peering routes are highlighted.

Subnet CIDR plan

Subnet	VPC	CIDR	Auto-assign public IPv4
ewb-a-public	VPC A	10.10.1.0/24	Off (public IP only on the bastion instance)
ewb-a-private	VPC A	10.10.2.0/24	Disabled
ewb-b-private	VPC B	10.20.1.0/24	Disabled

ewb-rt-a-public (rtb-041f1ac148cb6e157)

Associated subnet: **ewb-a-public (subnet-03019e9904066ce3d)**

Destination	Target	Purpose
10.10.0.0/16	local	Bastion to App-A
0.0.0.0/0	igw-0e4122ef5cbca2145	Bastion internet access
10.20.0.0/16	pcx-0aea73d442174951b	Bastion SSH to App-B

ewb-rt-a-private (rtb-00763f404aa0caec3)

Associated subnet: **ewb-a-private (subnet-05482c0832db2cc93)**

Destination	Target	Purpose
10.10.0.0/16	local	Existing VPC A route
	pcx-0aea73d442174951b	App-A to VPC B

ewb-rt-b-private (rtb-0a071720960f64016)

Associated subnet: **ewb-b-private (subnet-091ea124a7f25b03f)**

Destination	Target	Purpose
10.20.0.0/16	local	Existing VPC B route
	pcx-0aea73d442174951b	Return path to VPC A

> **Why the public subnet has a peering route**
> ProxyJump makes the bastion open its own connection to App-B, so the bastion subnet must know how to reach VPC B. The more specific 10.20.0.0/16 route is chosen over the 0.0.0.0/0 internet route.

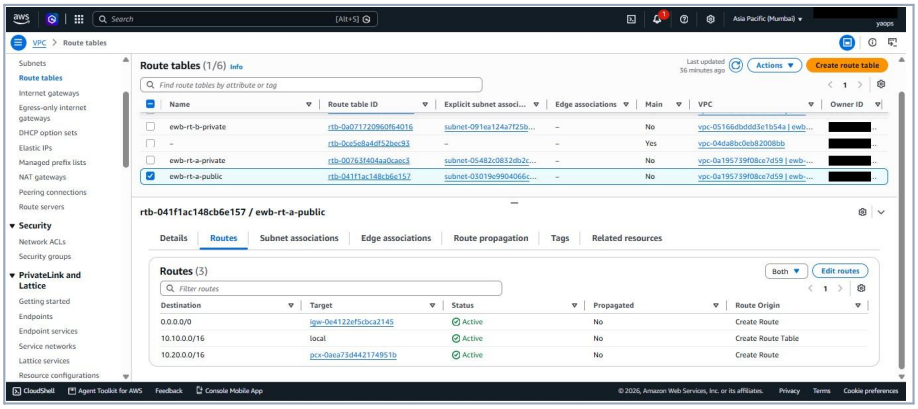
02b / ROUTING EVIDENCE

Route table screenshots

Checkpoint 04: routes and subnet associations for all three tables.

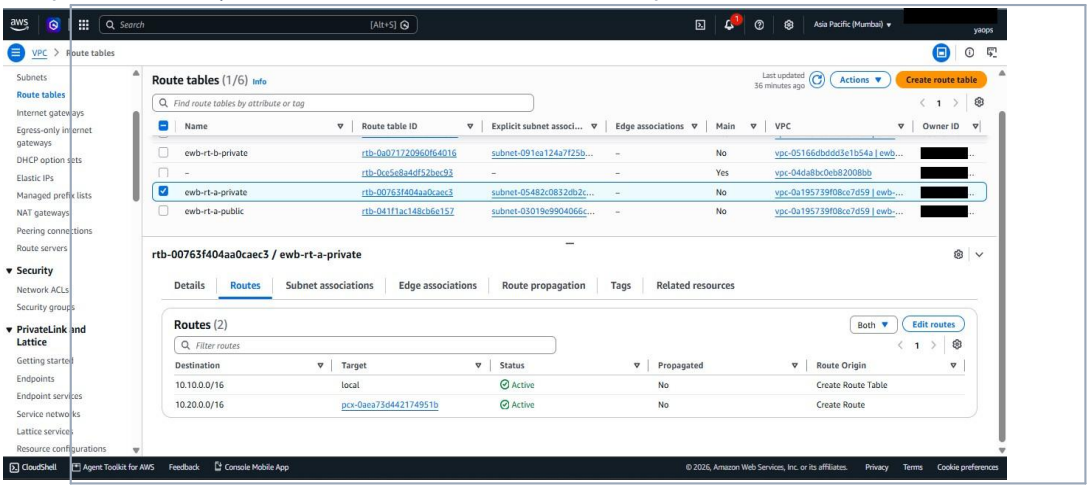
Host: VPC console | Test: ewb-rt-a-public routes

Expected result (acceptance criteria): 0.0.0.0/0 to igw, 10.10.0.0/16 local, 10.20.0.0/16 to pcx



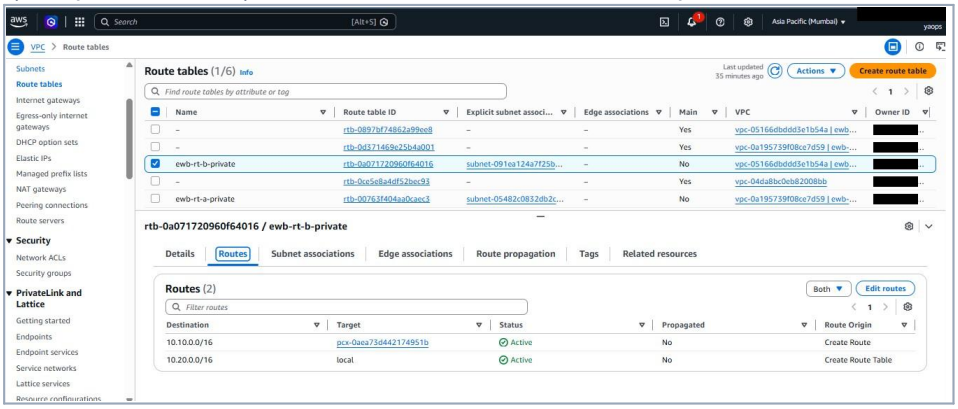
Host: VPC console | Test: ewb-rt-a-private routes

Expected result (acceptance criteria): 10.10.0.0/16 local, 10.20.0.0/16 to pcx



Host: VPC console | Test: ewb-rt-b-private routes

Expected result (acceptance criteria): 10.20.0.0/16 local, 10.10.0.0/16 to pcx



03 / PEERING AND SECURITY

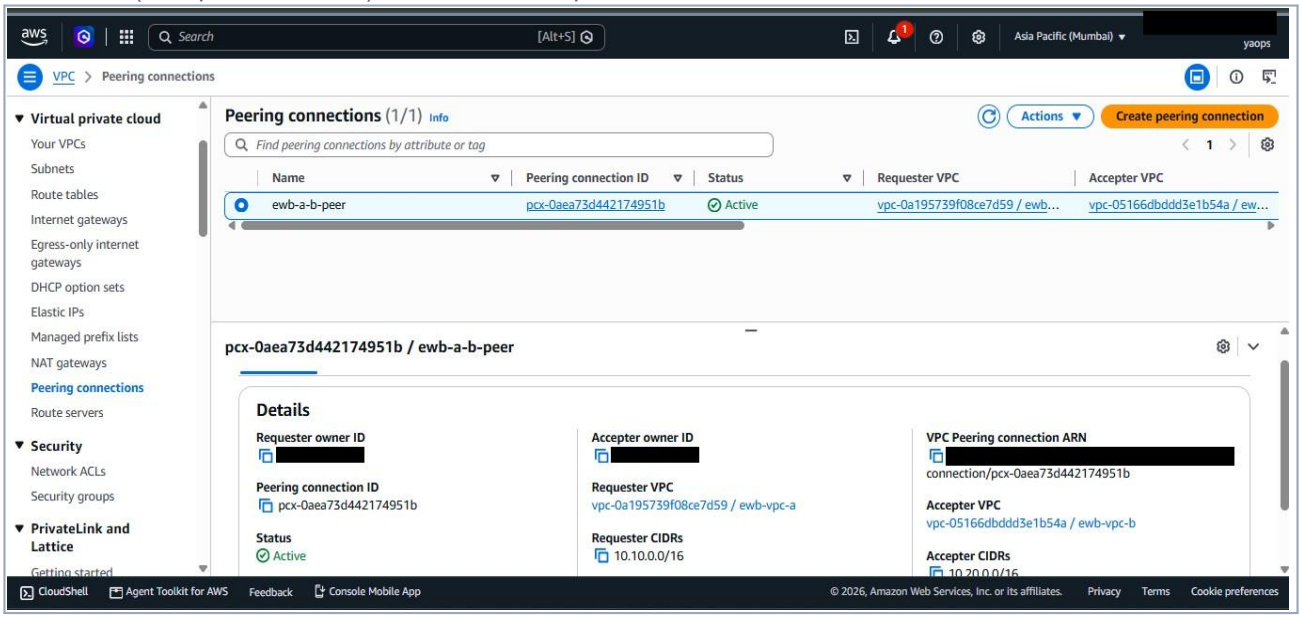
Peering connection and security group rules

Peering connection

Field	Value
Name	ewb-a-b-peer
Peering ID	pcx-0aea73d442174951b
Requester VPC	ewb-vpc-a / vpc-0a195739f08ce7d59 / 10.10.0.0/16
Accepter VPC	ewb-vpc-b / vpc-05166dbddd3e1b54a / 10.20.0.0/16
Account / Region	My account / ap-south-1 (same account and Region)
Status	Active (request accepted)

Host: VPC console | Test: Active peering connection

Expected result (acceptance criteria): Name ewb-a-b-peer, Status Active, both VPC IDs visible



Inbound security group rules

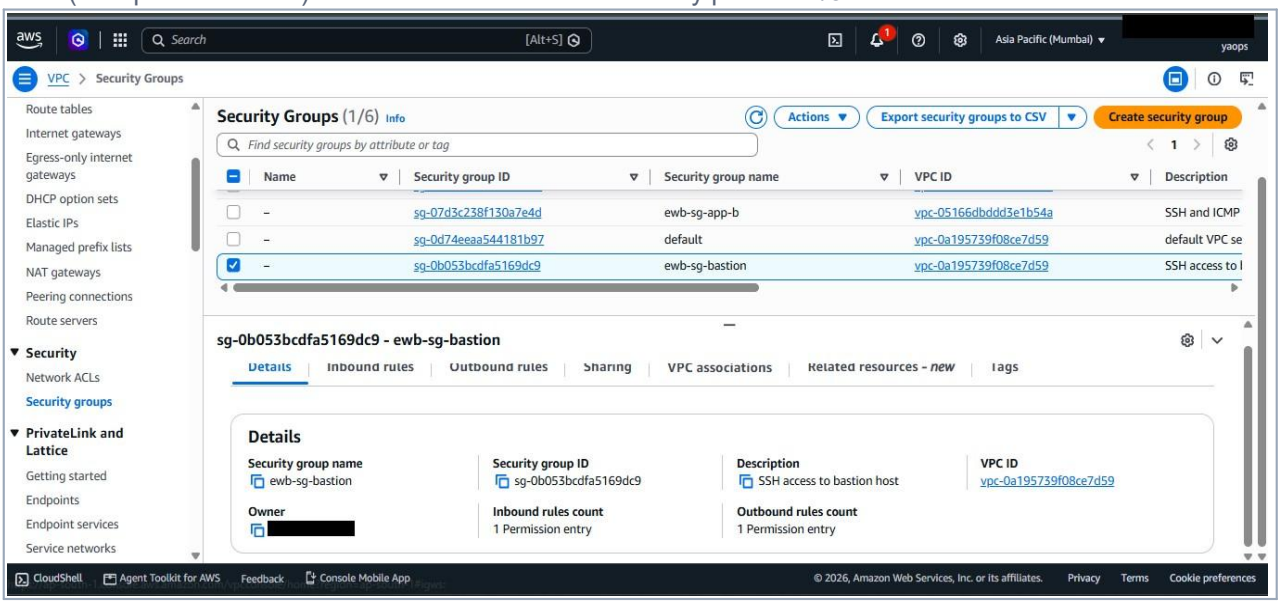
Security group	VPC	Type / port	Source
ewb-sg-bastion	VPC A	SSH / TCP 22	My current public IPv4 /32
ewb-sg-app-a	VPC A	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a	VPC A	ICMP Echo request (type 8)	10.20.1.10/32 (App-B)
ewb-sg-app-b	VPC B	SSH / TCP 22	10.10.1.10/32 (bastion private IP)

Security group	VPC	Type / port	Source
ewb-sg-app-b	VPC B	ICMP Echo request (type 8)	10.10.2.10/32 (App-A)

Outbound rules stay at the default allow-all. Security groups are stateful, so replies to permitted traffic are allowed automatically. Network ACLs are the default allow-all. SSH and ICMP are never opened to 0.0.0.0/0.

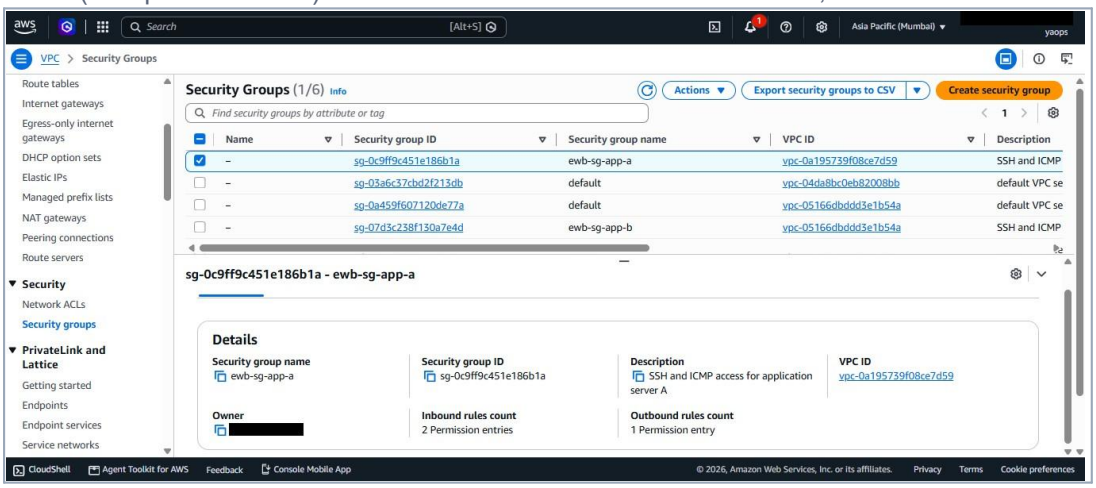
Host: VPC console | Test: ewb-sg-bastion (sg-0b053bcdfa5169dc9)

Expected result (acceptance criteria): 1 inbound rule: SSH 22 from my public IP /32



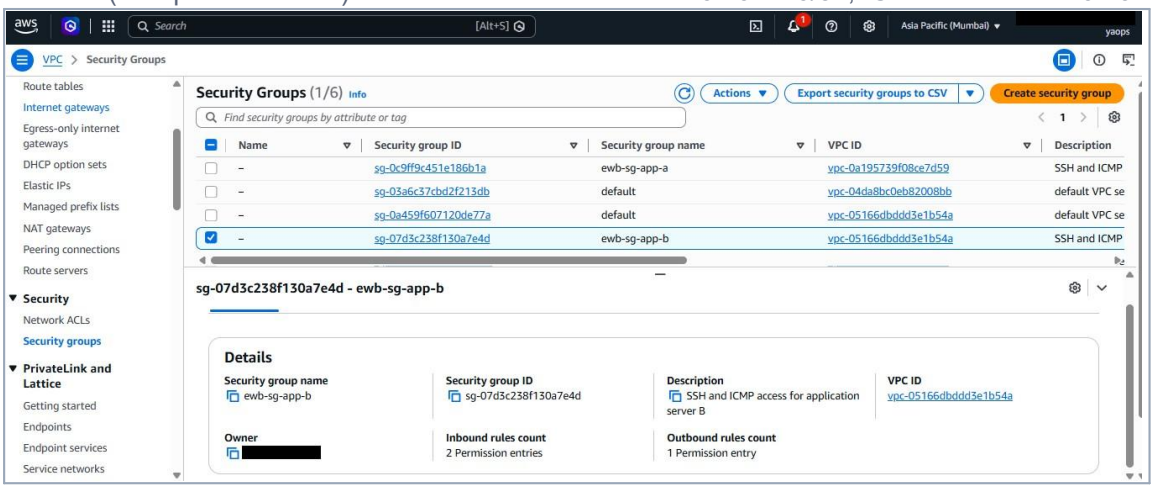
Host: VPC console | Test: ewb-sg-app-a (sg-0c9ff9c451e186b1a)

Expected result (acceptance criteria): 2 inbound rules: SSH from 10.10.1.10/32, ICMP Echo from 10.20.1.10/32



Host: VPC console | Test: ewb-sg-app-b (sg-07d3c238f130a7e4d)

Expected result (acceptance criteria): 2 inbound rules: SSH from 10.10.1.10/32, ICMP Echo from 10.10.2.10/32



04 / VERIFICATION

Connectivity tests

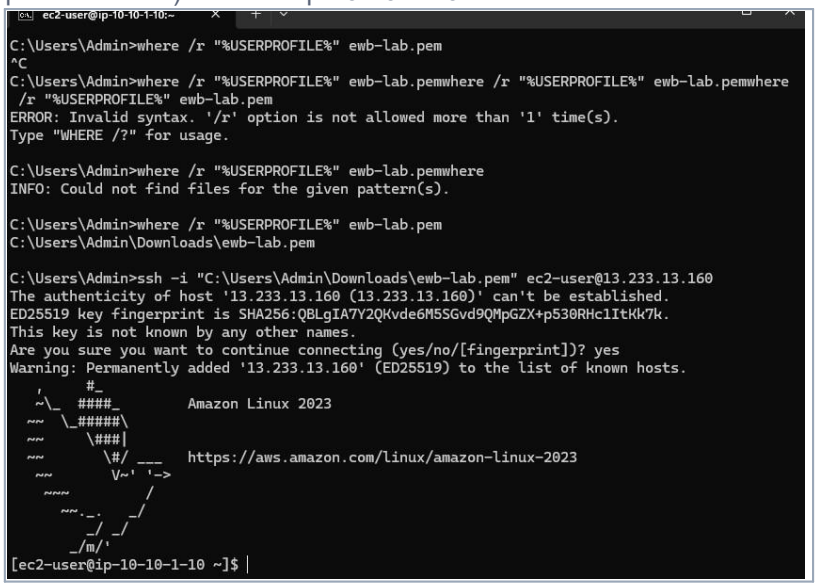
Labelled evidence for access, ping tests, private SSH and isolation.

Test summary

Test	Where / command	Expected result
00 Admin access	Laptop: SSH to bastion and App-A via ProxyJump	Shell on each host
01 Baseline	App-A, before peering: `ping -c 4 10.20.1.10`	No replies
02 Forward path	App-A, after routes: `ping -c 4 10.20.1.10`	4 replies, 0% packet loss
03 Reverse path	App-B, after routes: `ping -c 4 10.10.2.10`	4 replies, 0% packet loss
04 Private SSH	Laptop: SSH to App-B through the bastion	Shell on App-B through bastion
05 Isolation	EC2 instance details: App-A and App-B	No public IPv4 on either host

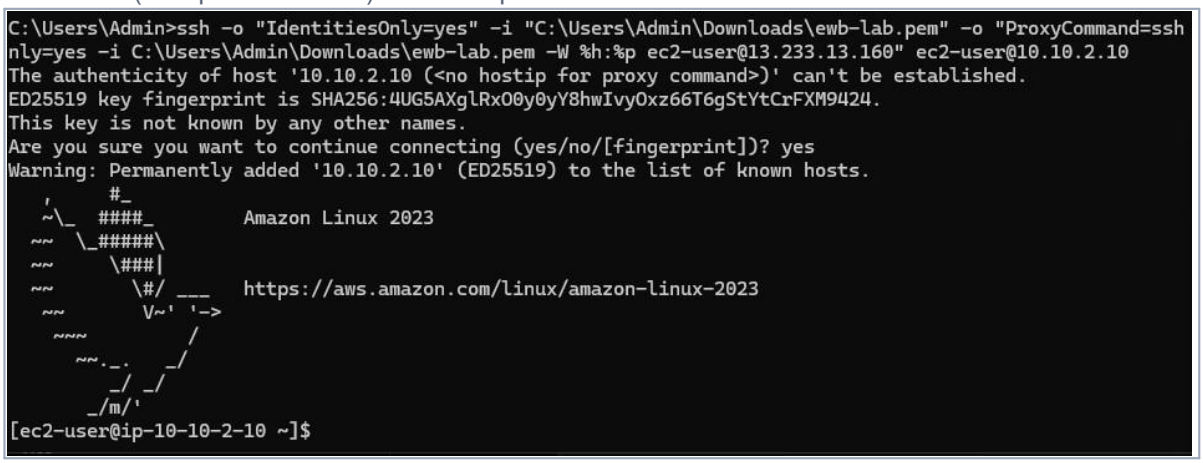
Host: Laptop to ewb-bastion (13.233.13.160) | Test: 00 SSH to the bastion

Expected result (acceptance criteria): Shell on ip-10-10-1-10



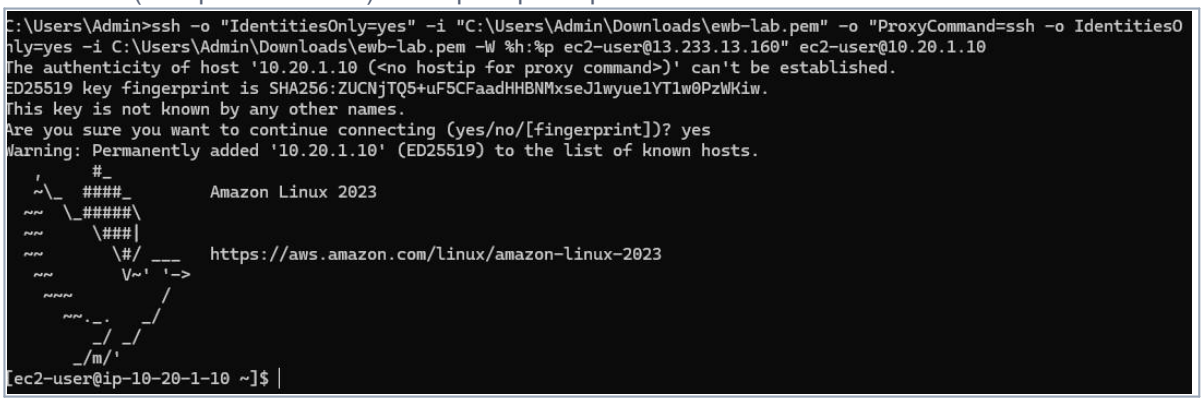
Host: Laptop to ewb-app-a (10.10.2.10) via bastion | Test: 00 SSH to App-A using ProxyJump

Expected result (acceptance criteria): Shell on ip-10-10-2-10



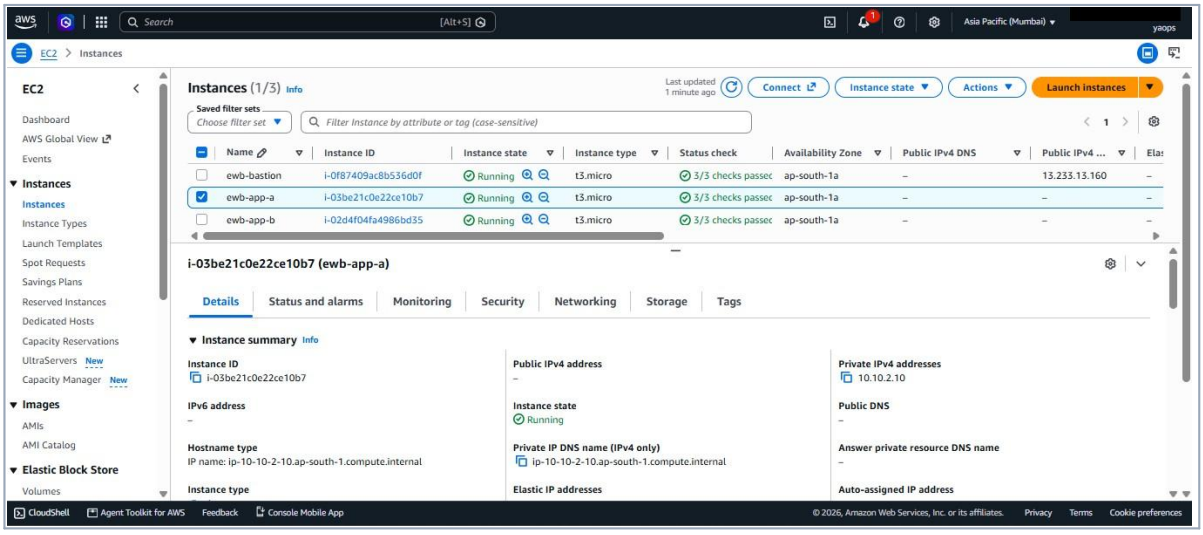
Host: Laptop to ewb-app-b (10.20.1.10) via bastion | Test: 04 Private SSH through the bastion

Expected result (acceptance criteria): Shell prompt on ip-10-20-1-10



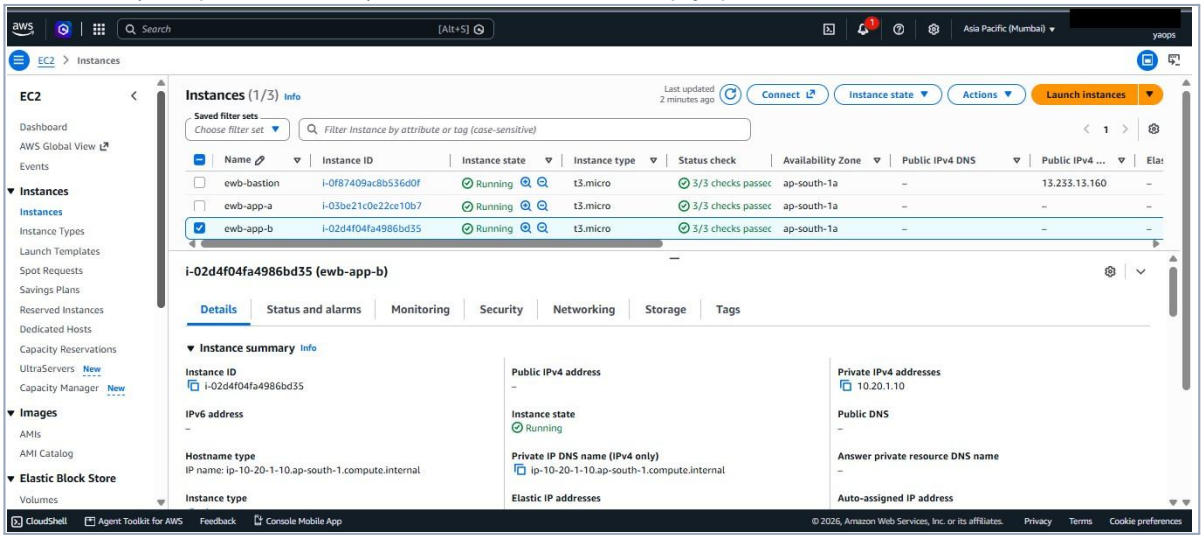
Host: EC2 console | Test: 05 Isolation: ewb-app-a instance details

Expected result (acceptance criteria): Public IPv4 address empty; private IP 10.10.2.10



Host: EC2 console | Test: 05 Isolation: ewb-app-b instance details

Expected result (acceptance criteria): Public IPv4 address empty; private IP 10.20.1.10



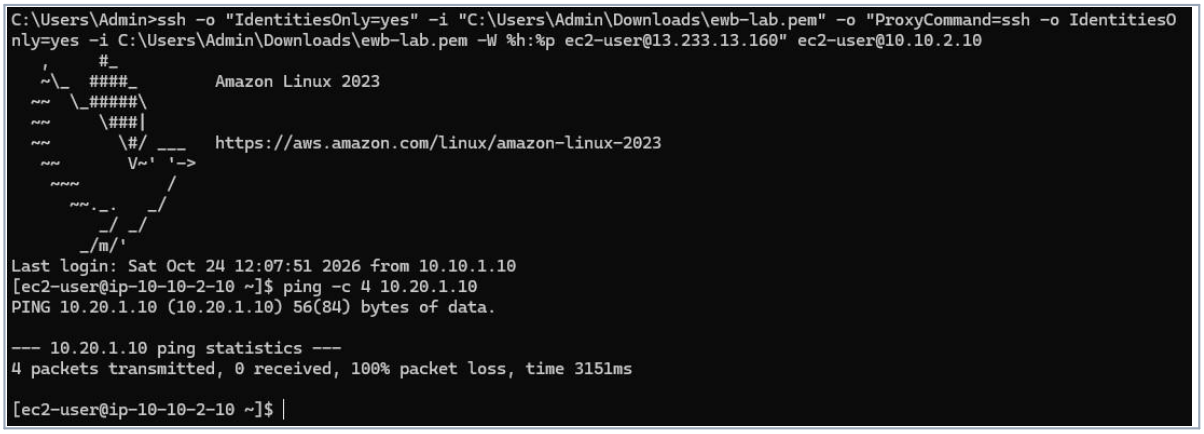
05 / CONTROLLED FAILURE

ICMP failure and recovery

Procedure: temporarily remove only the ewb-sg-app-b inbound ICMP Echo request rule (source 10.10.2.10/32), repeat the forward ping from App-A, then restore the exact rule and test again. The SSH rule stays in place.

Host: ewb-app-a (10.10.2.10) | Test: Ping 10.20.1.10 with the ICMP Echo rule removed from ewb-sg-app-b

Expected result (acceptance criteria): Replies stop (100% packet loss)



Explanation

Removing the rule stopped replies even though the peering connection was still Active and both route tables were correct. This shows that routing and security groups are separate layers: routes decide whether a packet can find a path, and the destination security group decides whether it is allowed in. Restoring the same rule brought connectivity back without changing anything else.

Forward path and return path

Forward path (App-A to App-B). App-A (10.10.2.10) sends a packet to 10.20.1.10. The route table ewb-rt-a-private matches 10.20.0.0/16 and sends it to the peering connection pcx-0aea73d442174951b. It reaches VPC B, and ewb-sg-app-b accepts it because the Echo request source 10.10.2.10/32 is allowed.

Return path (App-B to App-A). App-B replies to 10.10.2.10. The route table ewb-rt-b-private matches 10.10.0.0/16 and sends the reply back through the same peering connection. Because security groups are stateful, the reply is allowed automatically. Without this return route the request would arrive but the reply would be lost.

Bastion path to App-B. With ProxyJump, the bastion (10.10.1.10) opens the SSH connection to App-B. ewb-rt-a-public has 10.20.0.0/16 to the peering connection, which is more specific than 0.0.0.0/0, so the traffic uses peering. App-B sees the bastion private IP as the source, so ewb-sg-app-b allows SSH from 10.10.1.10/32.

06 / VIVA PREPARATION

Concept questions

Why is the return route needed?
Peering is only a link. Replies must also find a route back, so VPC B needs a route to 10.10.0.0/16 via the peering connection, otherwise ping and SSH replies are dropped.

Why is the App-B SSH source the bastion private IP?
ProxyJump makes the bastion open a new SSH connection to App-B, so App-B sees the bastion's private IP (10.10.1.10), not my laptop's public IP.

Why does Active peering not guarantee a successful ping?
Active only means the link exists. Route tables, the destination security group (ICMP Echo rule), host IPs and any network ACLs must all allow the traffic.

Can VPC B use VPC A's internet gateway over peering?
No. Peering is non-transitive and does not forward traffic to a gateway in the other VPC, so VPC B has no internet access through VPC A.

07 / CLEANUP

Cleanup evidence

Step	Action	Done
1	Terminated ewb-bastion, ewb-app-a, ewb-app-b; confirm lab EBS volumes are deleted; release any Elastic IP; remove unused key pair	[yes]
2	Deleted peer routes and ewb-a-b-peer; delete security groups, subnets, custom route tables; detach and delete ewb-igw-a; delete both VPCs	[yes]
3	Filter by tag Project = EWB-VPC-Peering and confirm EC2, volumes, public IPv4 and VPC resources are gone	[yes]

What I learned

Peering needs three things together: an active connection, routes on both sides, and security rules that allow the specific traffic. I also learned why non-overlapping CIDRs are required, why peering is non-transitive, and how a bastion with ProxyJump lets me manage private servers without copying my key onto any instance. *(Edit this note in your own words before submitting.)*